

SpiderFoot OSINT Integration with Wazuh

End-to-End Methodology, Validation, and Dashboard Report

Project: SpiderFoot OSINT → Wazuh SIEM/XDR Integration

Host: Ubuntu 24.04.4 LTS bare-metal endpoint

Hostname: flausino

Wazuh version: 4.14.5

Architecture: Wazuh Manager, Indexer, Dashboard, and Filebeat installed on the same host

SpiderFoot binding: 127.0.0.1:5002

Final dashboard: SpiderFoot OSINT Validation Dashboard

Report language: English

Audience: SOC analysts, detection engineers, security recruiters, and reproducibility reviewers

1. Executive Summary

This project implemented and validated a local SpiderFoot-to-Wazuh OSINT correlation pipeline on a single Ubuntu endpoint running a full Wazuh all-in-one environment. The goal was to build a defensible, recruiter-attractive OSINT detection workflow using native Wazuh ingestion, custom rules, MITRE ATT&CK mapping, and an OpenSearch/Wazuh dashboard.

The final pipeline is:

SpiderFoot OSINT source
↓
Normalized JSONL events
↓
/var/log/spiderfoot/events.jsonl
↓
Wazuh localfile ingestion with log_format=json
↓
Built-in JSON decoder
↓
Custom Wazuh rules 113200-113205
↓
alerts.json
↓
Wazuh Indexer / OpenSearch
↓
OpenSearch Dashboards visualizations
↓
SpiderFoot OSINT Validation Dashboard

The integration was validated at multiple layers:

1. SpiderFoot installation and service health
2. HTTP Digest authentication
3. JSONL file creation and permissions
4. Wazuh ossec.conf localfile ingestion
5. Rule syntax validation
6. Individual wazuh-logtest validation for all custom rules
7. Real-file synthetic event ingestion
8. alerts.json verification
9. OpenSearch DevTools verification
10. Dashboard/report visualization

The final validated rule range was:

No custom decoder was required for the final implementation because the Wazuh `json` log format invokes the built-in JSON decoder. Earlier decoder-related attempts were reviewed and intentionally not used in the final pipeline.

2. Methodological Principles

The methodology followed four operational principles:

1. Back up important files before modifying them.
2. Avoid destructive one-shot scripts.
3. Validate rule behavior with `wazuh-logtest` before relying on dashboard output.
4. Verify indexed alerts in OpenSearch DevTools before building dashboards.

The integration was intentionally implemented using native Wazuh capabilities wherever possible. The selected ingestion method was Wazuh Manager localfile collection, not Filebeat shipping, because SpiderFoot and Wazuh were running on the same host and the objective was a simple, reproducible, Wazuh-native pipeline.

3. Environment Baseline

The host baseline confirmed:

```
Operating System: Ubuntu 24.04.4 LTS
Kernel: Linux 6.17.0-35-generic
Architecture: x86_64
Hostname: flausino
Wazuh version: v4.14.5
Wazuh type: server
Wazuh manager: active/running
```

The Wazuh deployment was an all-in-one environment:

```
Wazuh Manager
Wazuh Indexer
Wazuh Dashboard
Filebeat
```

The SpiderFoot service was installed locally and bound only to loopback:

```
127.0.0.1:5002
```

This binding was selected to avoid exposing the SpiderFoot web UI directly to the LAN or Internet.

4. Pre-Integration Reconnaissance and Conflict Review

Before deploying the final rules, the existing Wazuh environment was reviewed for conflicts.

Important observations:

```
/var/ossec/etc/ossec.conf already contained a SpiderFoot localfile reference.  
/var/ossec/etc/rules/local_rules_json.xml contained older SpiderFoot rules using program_name=spiderfoot.  
/var/ossec/etc/decoders/local_decoder.xml had an existing unrelated syslog-style SpiderFoot decoder.  
/var/ossec/etc/decoders/local_decoder.xml.backup.* contained older SpiderFoot JSON decoder attempts.  
Rule IDs 100600-100607 were already used by Falco in this environment.
```

Because IDs `100600-100607` belonged to Falco, the final SpiderFoot rules were not placed in that range. The final range selected was:

```
113200-113205
```

The older SpiderFoot rules in `local_rules_json.xml` were not used for the final JSONL integration because they matched `program_name=spiderfoot`, whereas the final implementation uses structured JSON events with:

```
data.integration = spiderfoot
```

5. Files and Artifacts Created or Used

5.1 SpiderFoot application directory

```
/opt/spiderfoot
```

Purpose:

```
Native SpiderFoot installation cloned from GitHub.
```

Important subcomponents:

```
/opt/spiderfoot/.venv  
/opt/spiderfoot/sf.py  
/opt/spiderfoot/requirements.txt
```

The application was installed using a Python virtual environment to isolate SpiderFoot dependencies from the system Python environment.

5.2 SpiderFoot service account

A dedicated system user was used:

```
spiderfoot
```

Purpose:

```
Run the SpiderFoot web UI without using root privileges.  
Own the SpiderFoot application directory.  
Own the SpiderFoot JSONL output path.
```

5.3 SpiderFoot JSONL output directory

```
/var/log/spiderfoot
```

Purpose:

```
Directory used to store normalized SpiderFoot events for Wazuh ingestion.
```

Final permissions:

```
Owner/group: spiderfoot:wazuh  
Directory mode: 750
```

5.4 SpiderFoot JSONL event file

```
/var/log/spiderfoot/events.jsonl
```

Purpose:

```
Line-delimited JSON event file consumed by Wazuh Logcollector.
```

Final permissions:

```
Owner/group: spiderfoot:wazuh  
File mode: 640
```

Security rationale:

```
The spiderfoot user can write events.  
The wazuh group can read events.  
Normal users cannot read the file without elevated privileges.
```

5.5 SpiderFoot Digest authentication file

```
/opt/spiderfoot/.spiderfoot/passwd
```

Purpose:

Stores SpiderFoot HTTP Digest credentials.

Security note:

The credential value must not be published in documentation, screenshots, or GitHub repositories.

Recommended documentation form:

```
sfadmin:<REDACTED>
```

5.6 SpiderFoot systemd service

Created file:

```
/etc/systemd/system/spiderfoot.service
```

Final content:

```
[Unit]
Description=SpiderFoot OSINT Web UI
After=network-online.target
Wants=network-online.target

[Service]
Type=simple
User=spiderfoot
Group=spiderfoot
WorkingDirectory=/opt/spiderfoot
ExecStart=/opt/spiderfoot/.venv/bin/python /opt/spiderfoot/sf.py -l 127.0.0.1:5002
Restart=on-failure
RestartSec=5
NoNewPrivileges=true
PrivateTmp=true

[Install]
WantedBy=multi-user.target
```

Security and operational rationale:

Runs as non-root.
Binds only to localhost.
Restarts on failure.
Uses PrivateTmp and NoNewPrivileges for basic hardening.
Starts after network-online.target.

5.7 Wazuh manager configuration

Edited/verified file:

```
/var/ossec/etc/ossec.conf
```

Relevant final SpiderFoot localfile block:

```
<localfile>  
  <log_format>json</log_format>  
  <location>/var/log/spiderfoot/events.jsonl</location>  
  <label key="@source">spiderfoot</label>  
  <only-future-events>yes</only-future-events>  
</localfile>
```

Purpose:

Instruct Wazuh Logcollector to monitor the SpiderFoot JSONL file.
Parse each line as JSON.
Attach source metadata.
Avoid replaying old events by using only-future-events.

Important note:

The final alert documents use `data.integration: spiderfoot` because the synthetic events themselves included the field `"integration": "spiderfoot"`. The `ossec.conf` label `"@source": "spiderfoot"` is still useful as collection metadata, but the dashboard filter was built primarily on `data.integration`.

5.8 Wazuh custom rule file

Created file:

```
/var/ossec/etc/rules/spiderfoot_rules.xml
```

Final permissions:

```
Owner/group: root:wazuh  
File mode: 640
```

Purpose:

```
Define custom Wazuh rules for normalized SpiderFoot OSINT findings.
```

Final rule file:

```
<group name="spiderfoot,osint,reconnaissance,">

  <rule id="113200" level="3">
    <decoded_as>json</decoded_as>
    <field name="integration">spiderfoot</field>
    <description>SpiderFoot: normalized OSINT event ingested</description>
    <group>spiderfoot,osint,reconnaissance,</group>
  </rule>

  <rule id="113201" level="10">
    <if_sid>113200</if_sid>
    <field name="event_type" type="pcre2">(?!)(credential|credentials|secret|token|password|leak|breach|exposed)</field>
    <description>SpiderFoot: possible exposed credential or secret found</description>
    <mitre>
      <id>T1589.001</id>
    </mitre>
    <group>spiderfoot,osint,reconnaissance,credential_exposure,threat_intel,</group>
  </rule>

  <rule id="113202" level="6">
    <if_sid>113200</if_sid>
    <field name="event_type" type="pcre2">(?!)(email|identity|person|employee|username|account)</field>
    <description>SpiderFoot: victim identity or email-related OSINT finding</description>
    <mitre>
      <id>T1589.002</id>
    </mitre>
    <group>spiderfoot,osint,reconnaissance,identity_exposure,</group>
  </rule>

  <rule id="113203" level="5">
    <if_sid>113200</if_sid>
    <field name="event_type" type="pcre2">(?!)(domain|dns|ip|netblock|asn|whois|certificate|cert|subdomain|infrastructure)
  </field>
    <description>SpiderFoot: victim infrastructure OSINT finding</description>
    <mitre>
      <id>T1590</id>
    </mitre>
```

```

    <id>T1596</id>
  </mitre>
  <group>spiderfoot,osint,reconnaissance,infrastructure_intel,</group>
</rule>

<rule id="113204" level="7">
  <if_sid>113200</if_sid>
  <field name="event_type" type="pcre2">(?i)
(open_technical_database|passive_dns|whois|certificate_transparency|scan_database|shodan|censys|crtsh)</field>
  <description>SpiderFoot: open technical database reconnaissance finding</description>
  <mitre>
    <id>T1596</id>
  </mitre>
  <group>spiderfoot,osint,reconnaissance,open_technical_databases,</group>
</rule>

<rule id="113205" level="8">
  <if_sid>113200</if_sid>
  <field name="event_type" type="pcre2">(?i)(active_scan|port_scan|vulnerability_scan|direct_probe|active_recon)</field>
  <description>SpiderFoot: active reconnaissance or probing module activity</description>
  <mitre>
    <id>T1595</id>
  </mitre>
  <group>spiderfoot,osint,reconnaissance,active_scanning,</group>
</rule>

</group>

```

5.9 Decoder files

No new decoder file was required in the final implementation.

Reviewed but not used:

```
/var/ossec/etc/decoders/local_decoder.xml  
/var/ossec/etc/decoders/local_decoder.xml.backup.*
```

Reason:

Wazuh already has a built-in JSON decoder. Because ossec.conf uses `<log_format>json</log_format>`, the events are decoded as JSON without a custom decoder.

Final design decision:

Do not create a custom SpiderFoot JSON decoder for this pipeline.
Use `<decoded_as>json</decoded_as>` in the rules.

This reduced complexity and avoided conflicts caused by older malformed or redundant decoder attempts.

6. SpiderFoot Installation Methodology

The installation script followed the sequence below.

```
set -euo pipefail

TS="$(date +%Y%m%d_%H%M%S)"

echo "=== 1) Pre-flight: confirm port 5002 ==="
if sudo ss -ltnup | grep -qE '[:.]5002[[:space:]]'; then
    echo "ERROR: port 5002 is already in use. Stop here."
    sudo ss -ltnup | grep -E '[:.]5002[[:space:]]'
    exit 1
fi

echo "=== 2) Backup old installation, if present ==="
if [ -d /opt/spiderfoot ]; then
    sudo mv /opt/spiderfoot "/opt/spiderfoot.backup.$TS"
    echo "Backup created: /opt/spiderfoot.backup.$TS"
fi

if systemctl list-unit-files | grep -q '^spiderfoot.service'; then
    sudo systemctl stop spiderfoot || true
    sudo cp -a /etc/systemd/system/spiderfoot.service "/etc/systemd/system/spiderfoot.service.backup.$TS" || true
fi

echo "=== 3) Base packages ==="
sudo apt update
sudo apt install -y git python3-venv python3-pip jq curl sqlite3

echo "=== 4) Service user ==="
if ! id spiderfoot >/dev/null 2>&1; then
    sudo adduser --system --group --home /opt/spiderfoot spiderfoot
fi

echo "=== 5) SpiderFoot installation via git + venv ==="
sudo mkdir -p /opt
sudo git clone https://github.com/smicallef/spiderfoot.git /opt/spiderfoot
sudo chown -R spiderfoot:spiderfoot /opt/spiderfoot
```

```
sudo -u spiderfoot bash -lc '  
cd /opt/spiderfoot  
python3 -m venv .venv  
. .venv/bin/activate  
pip install --upgrade pip wheel setuptools  
pip install -r requirements.txt  
'  
  
echo "=== 6) JSONL log for Wazuh ==="  
sudo mkdir -p /var/log/spiderfoot  
sudo touch /var/log/spiderfoot/events.jsonl  
sudo chown -R spiderfoot:wazuh /var/log/spiderfoot  
sudo chmod 750 /var/log/spiderfoot  
sudo chmod 640 /var/log/spiderfoot/events.jsonl  
  
echo "=== 7) SpiderFoot Digest credentials ==="  
sudo -u spiderfoot bash -lc '  
mkdir -p ~/.spiderfoot  
printf "%s\n" "sfadmin:<REDACTED>" > ~/.spiderfoot/passwd  
chmod 700 ~/.spiderfoot  
chmod 600 ~/.spiderfoot/passwd  
'  
  
echo "=== 8) systemd service ==="  
sudo tee /etc/systemd/system/spiderfoot.service >/dev/null <<'EOF'  
[Unit]  
Description=SpiderFoot OSINT Web UI  
After=network-online.target  
Wants=network-online.target  
  
[Service]  
Type=simple  
User=spiderfoot  
Group=spiderfoot  
WorkingDirectory=/opt/spiderfoot  
ExecStart=/opt/spiderfoot/.venv/bin/python /opt/spiderfoot/sf.py -l 127.0.0.1:5002  
Restart=on-failure
```

```
RestartSec=5
NoNewPrivileges=true
PrivateTmp=true

[Install]
WantedBy=multi-user.target
EOF

sudo systemctl daemon-reload
sudo systemctl enable --now spiderfoot

echo "=== 9) Status and HTTP Digest test ==="
sleep 3
systemctl --no-pager --full status spiderfoot | sed -n '1,35p'

echo
echo "Expected HTTP code: 200"
curl --digest -u 'sfadmin:<REDACTED>' -s -o /dev/null -w '%{http_code}\n' http://127.0.0.1:5002/
```

Observed service result:

```
spiderfoot.service loaded and enabled.
spiderfoot.service active/running.
SpiderFoot started at 127.0.0.1:5002.
Authentication enabled based on supplied passwd file.
HTTP Digest test returned 200.
```

7. Wazuh Configuration Methodology

The Wazuh Manager was configured to ingest the SpiderFoot JSONL event file through `ossec.conf` .

Final localfile block:

```
<localfile>
  <log_format>json</log_format>
  <location>/var/log/spiderfoot/events.jsonl</location>
  <label key="@source">spiderfoot</label>
  <only-future-events>yes</only-future-events>
</localfile>
```

Validation objective:

Wazuh Logcollector must read /var/log/spiderfoot/events.jsonl.
Each JSON line must be decoded by the built-in JSON decoder.
The resulting structured fields must be available to rules as data.* fields in indexed alerts.

The choice of `only-future-events` was intentional. It prevents old JSONL data from being replayed into Wazuh after service restarts.

8. Rule Strategy

The rule strategy was hierarchical:

```
113200 = Base rule
113201-113205 = Child rules
```

8.1 Rule 113200 — Base event identification

Purpose:

Detect any normalized SpiderFoot event where integration=spiderfoot.

Level:

3

Detection logic:

decoded_as=json

integration=spiderfoot

This rule proves that Wazuh can parse and classify normalized SpiderFoot JSONL events.

8.2 Rule 113201 — Credential or secret exposure

Purpose:

Detect events related to exposed credentials, secrets, tokens, passwords, leaks, or breaches.

Level:

10

MITRE ATT&CK:

T1589.001 - Credentials

Detection field:

event_type

8.3 Rule 113202 — Identity or email exposure

Purpose:

Detect events related to email addresses, identity, usernames, people, employee exposure, or account references.

Level:

6

MITRE ATT&CK:

T1589.002 - Email Addresses

Detection field:

event_type

8.4 Rule 113203 — Infrastructure intelligence

Purpose:

Detect DNS, domain, IP, netblock, ASN, WHOIS, certificate, subdomain, or infrastructure-related findings.

Level:

5

MITRE ATT&CK:

T1590 - Gather Victim Network Information

T1596 - Search Open Technical Databases

Detection field:

event_type

8.5 Rule 113204 — Open technical database reconnaissance

Purpose:

Detect findings from open technical databases such as certificate transparency, passive DNS, WHOIS, Shodan, Censys, crt.sh, or scan databases.

Level:

7

MITRE ATT&CK:

T1596 - Search Open Technical Databases

Detection field:

event_type

8.6 Rule 113205 — Active reconnaissance

Purpose:

Detect active scan, port scan, vulnerability scan, direct probe, or active reconnaissance events.

Level:

8

MITRE ATT&CK:

T1595 - Active Scanning

Detection field:

event_type

9. Wazuh Configuration Validation

Before restarting Wazuh Manager, the configuration was validated with:

```
sudo /var/ossec/bin/wazuh-analysisd -t
```

Expected result:

```
No fatal syntax error.  
Rules loaded successfully.  
Configuration safe for Wazuh Manager restart.
```

After validation, Wazuh Manager was restarted:

```
sudo systemctl restart wazuh-manager
```

10. wazuh-logtest Validation

Every custom SpiderFoot rule was validated with `wazuh-logtest` .

Command:

```
sudo /var/ossec/bin/wazuh-logtest
```

The methodology was to test one JSON event per category.

10.1 Rule 113200 test — Generic normalized event

Test input:

```
{"integration":"spiderfoot","event_type":"generic","module":"sfp_test","data_type":"DOMAIN_NAME","message":"SpiderFoot Wazuh  
real-file ingestion test","target":"example.com"}
```

Expected result:

Rule ID: 113200
Level: 3
Description: SpiderFoot: normalized OSINT event ingested
Decoded as: json

Validation conclusion:

The base rule correctly identified a normalized SpiderFoot JSON event.

10.2 Rule 113201 test — Credential exposure

Test input:

```
{"integration":"spiderfoot","event_type":"credential_leak","module":"sfp_hibp","data_type":"PASSWORD_COMPROMISED","message":  
"Possible exposed password found in breach data","target":"example.com"}
```

Expected result:

Rule ID: 113201
Level: 10
Description: SpiderFoot: possible exposed credential or secret found
MITRE ID: T1589.001
MITRE technique: Credentials

Validation conclusion:

Credential and breach-related OSINT events correctly escalated to high severity.

10.3 Rule 113202 test — Email or identity exposure

Test input:

```
{"integration":"spiderfoot","event_type":"email_identity","module":"sfp_email","data_type":"EMAILADDR","message":"Public email address discovered","target":"example.com"}
```

Expected result:

```
Rule ID: 113202
Level: 6
Description: SpiderFoot: victim identity or email-related OSINT finding
MITRE ID: T1589.002
MITRE technique: Email Addresses
```

Validation conclusion:

Email and identity-related OSINT events were correctly classified.

10.4 Rule 113203 test — Infrastructure intelligence

Test input:

```
{"integration":"spiderfoot","event_type":"dns_infrastructure","module":"sfp_dns","data_type":"SUBDOMAIN","message":"Subdomain discovered","target":"example.com"}
```

Expected result:

```
Rule ID: 113203
Level: 5
Description: SpiderFoot: victim infrastructure OSINT finding
MITRE IDs: T1590, T1596
MITRE techniques: Gather Victim Network Information, Search Open Technical Databases
```

Validation conclusion:

```
Infrastructure-related reconnaissance findings were correctly classified and mapped to MITRE ATT&CK.
```

10.5 Rule 113204 test — Open technical database

Test input:

```
{"integration":"spiderfoot","event_type":"open_technical_database","module":"sfp_crtsh","data_type":"SSL_CERTIFICATE","message":"Certificate transparency result found","target":"example.com"}
```

Expected result:

Rule ID: 113204
Level: 7
Description: SpiderFoot: open technical database reconnaissance finding
MITRE ID: T1596
MITRE technique: Search Open Technical Databases

Validation conclusion:

Open technical database reconnaissance events were correctly detected.

10.6 Rule 113205 test — Active reconnaissance

Test input:

```
{"integration":"spiderfoot","event_type":"active_scan","module":"sfp_portscan","data_type":"TCP_PORT_OPEN","message":"Active probe found exposed TCP port","target":"example.com"}
```

Expected result:

Rule ID: 113205
Level: 8
Description: SpiderFoot: active reconnaissance or probing module activity
MITRE ID: T1595
MITRE technique: Active Scanning

Validation conclusion:

Active reconnaissance events were correctly detected and mapped to MITRE ATT&CK.

11. Synthetic Event Generation

After rule validation, controlled synthetic events were appended to the real SpiderFoot JSONL file to validate the complete Wazuh pipeline.

Command sequence:

```
sudo /var/ossec/bin/wazuh-analysisd -t && sudo systemctl restart wazuh-manager

sleep 15

NOW="$(date -Iseconds)"

sudo -u spiderfoot tee -a /var/log/spiderfoot/events.jsonl <<EOF
{"integration":"spiderfoot","event_type":"generic","module":"sfp_test","data_type":"DOMAIN_NAME","message":"SpiderFoot Wazuh
real-file ingestion test","target":"example.com","timestamp":"$NOW"}
{"integration":"spiderfoot","event_type":"credential_leak","module":"sfp_hibp","data_type":"PASSWORD_COMPROMISED","message":
"Possible exposed password found in breach data","target":"example.com","timestamp":"$NOW"}
{"integration":"spiderfoot","event_type":"email_identity","module":"sfp_email","data_type":"EMAILADDR","message":"Public
email address discovered","target":"example.com","timestamp":"$NOW"}
{"integration":"spiderfoot","event_type":"dns_infrastructure","module":"sfp_dns","data_type":"SUBDOMAIN","message":"Subdomai
n discovered","target":"example.com","timestamp":"$NOW"}
{"integration":"spiderfoot","event_type":"open_technical_database","module":"sfp_crtsh","data_type":"SSL_CERTIFICATE","messa
ge":"Certificate transparency result found","target":"example.com","timestamp":"$NOW"}
{"integration":"spiderfoot","event_type":"active_scan","module":"sfp_portscan","data_type":"TCP_PORT_OPEN","message":"Active
probe found exposed TCP port","target":"example.com","timestamp":"$NOW"}
EOF

sleep 10
```

Purpose:

Generate controlled events for every custom rule.
Validate real-file ingestion through Wazuh Logcollector.
Confirm that events reach alerts.json and the Wazuh Indexer.

12. Local alerts.json Verification

The first verification layer after real-file ingestion was:

```
sudo jq -c '
  select(.data.integration=="spiderfoot")
  | {
    ts: .timestamp,
    rule: .rule.id,
    level: .rule.level,
    desc: .rule.description,
    event_type: .data.event_type,
    module: .data.module,
    mitre: .rule.mitre.id
  }
' /var/ossec/logs/alerts/alerts.json | tail -n 20
```

Observed note:

jq occasionally returned "parse error: Unfinished string at EOF" because alerts.json was being read while Wazuh was actively writing a partial JSON line.

SOC interpretation:

This does not invalidate the pipeline. The same command still returned valid SpiderFoot alerts, and the authoritative verification was later performed through OpenSearch DevTools.

Local validation showed that the following rules appeared:

```
113200
113201
113202
113203
113204
113205
```

13. OpenSearch DevTools Verification

OpenSearch DevTools was used as the authoritative index-layer verification.

13.1 Query: retrieve indexed SpiderFoot alerts

```
GET wazuh-alerts-*/_search
```

```
{
  "size": 20,
  "sort": [
    {
      "@timestamp": {
        "order": "desc"
      }
    }
  ],
  "_source": [
    "@timestamp",
    "data.integration",
    "data.event_type",
    "data.module",
    "data.data_type",
    "data.message",
    "data.target",
    "rule.id",
    "rule.level",
    "rule.description",
    "rule.mitre.id",
    "rule.mitre.technique",
    "location"
  ],
  "query": {
    "bool": {
      "filter": [
        {
          "term": {
            "data.integration": "spiderfoot"
          }
        }
      ]
    }
  }
}
```

```
}  
}
```

Observed indexed fields:

```
@timestamp  
data.integration  
data.event_type  
data.module  
data.data_type  
data.message  
data.target  
rule.id  
rule.level  
rule.description  
rule.mitre.id  
rule.mitre.technique  
location
```

Representative document structure:

```
{
  "@timestamp": "2026-06-15T14:19:33.727Z",
  "data": {
    "event_type": "credential_leak",
    "module": "sfp_hibp",
    "integration": "spiderfoot",
    "data_type": "PASSWORD_COMPROMISED",
    "message": "Possible exposed password found in breach data",
    "target": "example.com"
  },
  "rule": {
    "level": 10,
    "description": "SpiderFoot: possible exposed credential or secret found",
    "mitre": {
      "technique": [
        "Credentials"
      ],
      "id": [
        "T1589.001"
      ],
      "tactic": [
        "Reconnaissance"
      ]
    },
    "id": "113201"
  },
  "location": "/var/log/spiderfoot/events.jsonl"
}
```

13.2 Query: aggregate by custom rule

```
GET wazuh-alerts-*/_search
{
  "size": 0,
  "query": {
    "bool": {
      "filter": [
        {
          "term": {
            "data.integration": "spiderfoot"
          }
        }
      ]
    }
  },
  "aggs": {
    "spiderfoot_rules": {
      "terms": {
        "field": "rule.id",
        "size": 20,
        "order": {
          "_key": "asc"
        }
      }
    }
  }
}
```

Observed result:

Total SpiderFoot alerts: 42

113200: 7

113201: 7

113202: 7

113203: 7

113204: 7

113205: 7

SOC interpretation:

All six custom rules fired successfully and uniformly during controlled validation.

13.3 Query: aggregate by event type

```
GET wazuh-alerts-*/_search
{
  "size": 0,
  "query": {
    "bool": {
      "filter": [
        {
          "term": {
            "data.integration": "spiderfoot"
          }
        }
      ]
    }
  },
  "aggs": {
    "event_types": {
      "terms": {
        "field": "data.event_type",
        "size": 20
      }
    }
  }
}
```

Observed result:

```
active_scan: 7
credential_leak: 7
dns_infrastructure: 7
email_identity: 7
generic: 7
open_technical_database: 7
```

SOC interpretation:

Each normalized OSINT event class was successfully ingested, decoded, matched, alerted, and indexed.

13.4 Query: aggregate by MITRE technique

GET wazuh-alerts-*/_search

```
{
  "size": 0,
  "query": {
    "bool": {
      "filter": [
        {
          "term": {
            "data.integration": "spiderfoot"
          }
        },
        {
          "exists": {
            "field": "rule.mitre.id"
          }
        }
      ]
    }
  },
  "aggs": {
    "mitre_ids": {
      "terms": {
        "field": "rule.mitre.id",
        "size": 20
      }
    },
    "mitre_techniques": {
      "terms": {
        "field": "rule.mitre.technique",
        "size": 20
      }
    }
  }
}
```

Observed MITRE ID result:

```
T1596: 14
T1589.001: 7
T1589.002: 7
T1590: 7
T1595: 7
```

Observed MITRE technique result:

```
Search Open Technical Databases: 14
Active Scanning: 7
Credentials: 7
Email Addresses: 7
Gather Victim Network Information: 7
```

SOC interpretation:

T1596 appears with a higher count because it is mapped in more than one rule, specifically infrastructure intelligence and open technical database reconnaissance.

13.5 Query: aggregate by severity level

```
GET wazuh-alerts-*/_search
{
  "size": 0,
  "query": {
    "bool": {
      "filter": [
        {
          "term": {
            "data.integration": "spiderfoot"
          }
        }
      ]
    }
  },
  "aggs": {
    "severity_levels": {
      "terms": {
        "field": "rule.level",
        "size": 20,
        "order": {
          "_key": "asc"
        }
      }
    }
  }
}
```

Observed result:

```
3: 7
5: 7
6: 7
7: 7
8: 7
10: 7
```

SOC interpretation:

```
The custom severity model behaved as designed. Each event class mapped to a different operational severity level.
```

14. Dashboard Design

The final dashboard was designed with five panels:

1. SpiderFoot OSINT Alerts Over Time
2. SpiderFoot Alerts by Detection Rule
3. SpiderFoot Alert Severity Distribution
4. SpiderFoot Findings by Event Type
5. SpiderFoot Findings Mapped to MITRE ATT&CK

Recommended dashboard-level DQL filter:

```
data.integration: "spiderfoot"
```

Alternative stricter filter:

```
data.integration: "spiderfoot" AND rule.id: (113200 OR 113201 OR 113202 OR 113203 OR 113204 OR 113205)
```

Recommended time range:

Last 24 hours, Last 30 days, or the exact validation time window.

For reproducible portfolio screenshots, the dashboard should use the exact validation window or a saved DQL filter to ensure only SpiderFoot validation events are shown.

15. Final Visualization Specifications

15.1 Panel 1 — SpiderFoot OSINT Alerts Over Time

Visualization type:

Vertical bar chart

Title:

SpiderFoot OSINT Alerts Over Time

Description:

Timeline of normalized SpiderFoot OSINT validation alerts ingested by Wazuh from `/var/log/spiderfoot/events.jsonl`.

DQL filter:

`data.integration: "spiderfoot"`

Y-axis:

Aggregation: Count

Custom label: Count or Alert count

X-axis:

Aggregation: Date histogram

Field: `@timestamp`

Interval: Fixed interval

Value: 1m

Custom label: Event time

Expected interpretation:

Shows the temporal ingestion pattern of SpiderFoot validation events.

SOC note:

If this panel shows "All docs" or an unusually high count, the panel or dashboard filter is not restricted to SpiderFoot events.

15.2 Panel 2 — SpiderFoot Alerts by Detection Rule

Visualization type:
Horizontal bar chart

Title:
SpiderFoot Alerts by Detection Rule

Description:
Distribution of SpiderFoot OSINT validation alerts across custom Wazuh detection rules 113200-113205.

DQL filter:
data.integration: "spiderfoot"

Y-axis:
Aggregation: Count
Custom label: Alert count

X-axis:
Aggregation: Filters

Filters:
rule.id: "113200"
Label: 113200 - Generic OSINT event

rule.id: "113201"
Label: 113201 - Credential or secret exposure

rule.id: "113202"
Label: 113202 - Identity or email exposure

rule.id: "113203"
Label: 113203 - Infrastructure intelligence

rule.id: "113204"
Label: 113204 - Open technical database finding

```
rule.id: "113205"  
Label: 113205 - Active reconnaissance
```

Expected result:

```
Each rule bucket should show 7 alerts in the controlled validation dataset.
```

15.3 Panel 3 — SpiderFoot Alert Severity Distribution

Visualization type:

Vertical bar chart

Title:

SpiderFoot Alert Severity Distribution

Description:

Severity distribution of SpiderFoot OSINT validation alerts based on custom Wazuh rule levels.

DQL filter:

data.integration: "spiderfoot"

Y-axis:

Aggregation: Count

Custom label: Alert count

X-axis:

Aggregation: Filters

Filters:

rule.level: 3

Label: 3 - Generic OSINT event

rule.level: 5

Label: 5 - Infrastructure intelligence

rule.level: 6

Label: 6 - Identity or email exposure

rule.level: 7

Label: 7 - Open technical database finding

rule.level: 8

Label: 8 - Active reconnaissance

```
rule.level: 10
```

```
Label: 10 - Possible credential or secret exposure
```

Expected result:

Each severity bucket should show 7 alerts in the controlled validation dataset.

15.4 Panel 4 — SpiderFoot Findings by Event Type

Visualization type:

Vertical bar chart

Title:

SpiderFoot Findings by Event Type

Description:

Breakdown of normalized SpiderFoot finding categories generated during controlled OSINT validation.

DQL filter:

data.integration: "spiderfoot"

Y-axis:

Aggregation: Count

Custom label: Finding count

X-axis:

Aggregation: Filters

Filters:

data.event_type: "credential_leak"

Label: Credential leak

data.event_type: "email_identity"

Label: Email / identity exposure

data.event_type: "dns_infrastructure"

Label: DNS / infrastructure intelligence

data.event_type: "open_technical_database"

Label: Open technical database finding

data.event_type: "active_scan"

Label: Active reconnaissance

```
data.event_type: "generic"  
Label: Generic OSINT event
```

Expected result:

```
Each event type should show 7 findings in the controlled validation dataset.
```

15.5 Panel 5 — SpiderFoot Findings Mapped to MITRE ATT&CK

Visualization type:

Vertical or horizontal bar chart

Title:

SpiderFoot Findings Mapped to MITRE ATT&CK

Description:

SpiderFoot OSINT validation alerts mapped to MITRE ATT&CK reconnaissance techniques.

DQL filter:

data.integration: "spiderfoot" AND rule.mitre.id: *

Y-axis:

Aggregation: Count

Custom label: Alert count

X-axis:

Aggregation: Terms

Field: rule.mitre.technique

Size: 10

Order by: Count

Order: Descending

Custom label: MITRE ATT&CK technique

Expected SpiderFoot-only techniques:

Search Open Technical Databases

Active Scanning

Credentials

Email Addresses

Gather Victim Network Information

Expected result:

Search Open Technical Databases should have the highest count because T1596 appears in more than one SpiderFoot rule.

SOC note:

If unrelated MITRE techniques such as Sudo and Sudo Caching, Command and Scripting Interpreter, File and Directory Discovery, Valid Accounts, or Create Account appear, then the visualization is not properly restricted to data.integration:spiderfoot.

16. Dashboard Screenshot Analysis

The final attached OpenSearch Dashboards report contains the intended five-panel layout:

Top-left:

SpiderFoot OSINT Alerts Over Time

Top-right:

SpiderFoot Alerts by Detection Rule

Middle-left:

SpiderFoot Alert Severity Distribution

Middle-right:

SpiderFoot Findings by Event Type

Bottom:

SpiderFoot Findings Mapped to MITRE ATT&CK

Positive observations:

The dashboard structure is correct.
The detection-rule panel shows the six SpiderFoot rule categories.
The severity panel shows the six intended severity categories.
The event-type panel shows the normalized SpiderFoot event categories.
The dashboard is suitable as a final portfolio artifact after filter verification.

Quality-control observations:

The report screenshot shows an empty global search bar.
The time-series panel appears to show "All docs", which may indicate the SpiderFoot DQL filter was not applied or not saved at the dashboard level.
The MITRE ATT&CK panel appears to include non-SpiderFoot techniques, which indicates that this panel should be checked for a saved panel-level filter or a dashboard-level DQL filter.

Required final dashboard QA before publication:

1. Confirm that the dashboard-level query is:
data.integration: "spiderfoot"
 2. Confirm that the time-series panel does not show "All docs" as an unfiltered bucket.
 3. Confirm that the MITRE ATT&CK panel only shows SpiderFoot-related techniques:
Search Open Technical Databases
Active Scanning
Credentials
Email Addresses
Gather Victim Network Information
 4. Regenerate the OpenSearch report after saving the dashboard with the correct global or panel-level filters.
-

17. Security and Privacy Considerations

The project included a personal OSINT audit phase, but personal OSINT findings were intentionally kept separate from the Wazuh portfolio dataset.

Security decisions:

Personal email addresses were not intentionally indexed into Wazuh for portfolio use.
Adult-site or sensitive personal findings were not sent to Wazuh.
Synthetic example.com events were used for validation.
Dashboard evidence was based on controlled validation data.

This is the correct approach for a public portfolio:

Use synthetic or redacted data.
Do not expose personal emails.
Do not expose adult-site account names.
Do not expose URLs from sensitive findings.
Do not publish raw ossec.conf containing API keys.
Do not publish SpiderFoot authentication credentials.

18. Final SOC Analyst Assessment

The SpiderFoot-to-Wazuh integration is technically successful.

Evidence:

SpiderFoot is installed natively under /opt/spiderfoot.
SpiderFoot runs as a systemd service.
SpiderFoot is bound to 127.0.0.1:5002.
HTTP Digest authentication works and returned HTTP 200.
The Wazuh-readable JSONL file exists at /var/log/spiderfoot/events.jsonl.
Wazuh ossec.conf contains a localfile JSON ingestion block.
No custom decoder was required.
The final custom rules are in /var/ossec/etc/rules/spiderfoot_rules.xml.
Rules 113200-113205 were individually validated with wazuh-logtest.
Synthetic events were written to the real JSONL file.
Wazuh generated alerts for every rule.
OpenSearch DevTools confirmed 42 indexed SpiderFoot alerts.
Aggregations confirmed 7 alerts per custom rule.
Aggregations confirmed 7 alerts per event type.
MITRE mappings were indexed for the relevant child rules.
The dashboard contains five final visualizations.

Final pipeline status:

Operationally validated.
Scientifically reproducible.
Suitable for portfolio documentation after final dashboard filter QA.

19. Recommended Repository Documentation Structure

Recommended GitHub structure:

```
spiderfoot-wazuh-integration/  
├─ README.md  
├─ docs/  
│   ├── methodology.md  
│   ├── validation-results.md  
│   ├── dashboard-design.md  
│   └─ security-notes.md  
├─ wazuh/  
│   ├── rules/  
│   │   └─ spiderfoot_rules.xml  
│   └─ ossec-conf-snippets/  
│       └─ spiderfoot-localfile.xml  
├─ scripts/  
│   └─ generate-spiderfoot-validation-events.sh  
├─ screenshots/  
│   └─ spiderfoot-osint-validation-dashboard.png  
└─ LICENSE
```

Files that should not be committed:

```
/opt/spiderfoot/.spiderfoot/passwd  
/var/ossec/etc/ossec.conf full raw copy  
Any file containing API keys  
Any personal OSINT result  
Any adult-site URL or account reference  
Any real email address from the audit
```

20. Conclusion

This project demonstrates a complete OSINT-to-SIEM detection workflow. SpiderFoot provides OSINT finding context, Wazuh performs ingestion and detection, custom rules classify findings by severity and MITRE ATT&CK technique, OpenSearch confirms indexed alert

evidence, and the dashboard visualizes the final detection pipeline.

The strongest technical contribution is not simply installing SpiderFoot, but converting SpiderFoot output into structured, validated, MITRE-mapped Wazuh detections through a controlled and reproducible SOC engineering workflow.